



Project Title: Build-To-Order (BTO) Management System

Declaration of Original Work for SC2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below. We have honored the principles of academic integrity and have upheld the Student Code of Academic Conduct in the completion of this work. We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course	Lab Group	Signature/Date
PEH JIA QIAN	SC2002	FDAB	Jiaqian 23/4/25
PRITHEEV ROSHAN S/O BASKARAN	SC2002	FDAB	Pritheev 23/4/25
RENEE TEO RUI NING	SC2002	FDAB	Renee 23/4/25
SONALI MOORTHY	SC2002	FDAB	Sonali23/4/25
TAN WEI JIN, RANEN	SC2002	FDAB	Ranen 23/4/25

GitHub Link: <https://github.com/Axieof/BTOSystem>

1. Introduction

For our BTO system, our goal is to ensure that our overall structure aligns with SOLID principles while using the Model-View-Controller (MVC) architecture. By separating each layer into its respective package, this forms the skeleton of the system, enhancing its maintainability, extensibility, and testability.

1.1 Explicit Requirements and Implicit Requirements

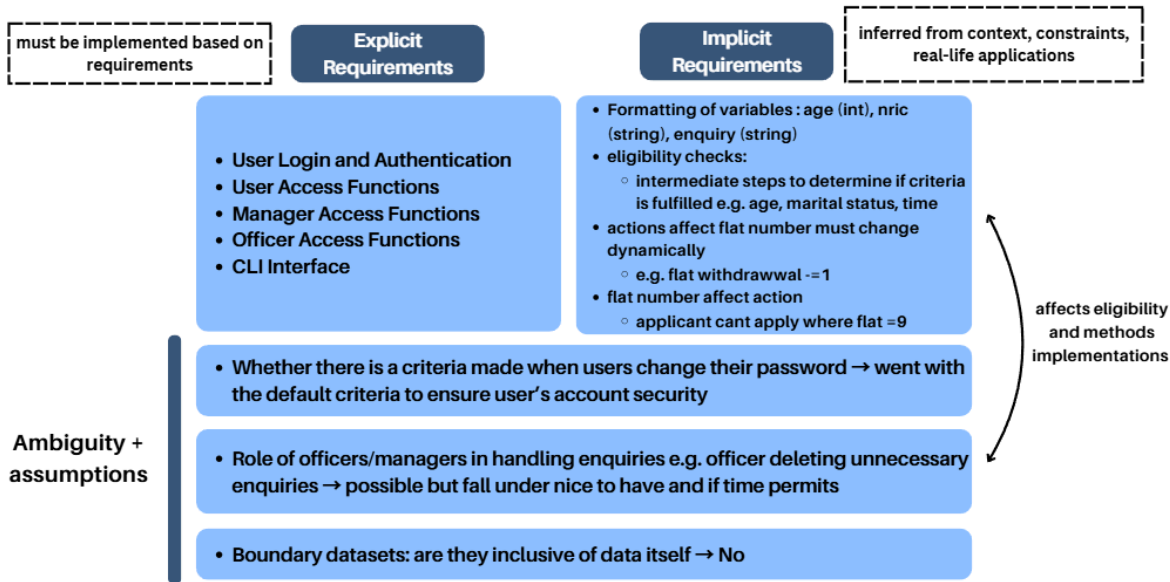


Figure 1. Implicit and Explicit Requirements, Assumptions and Ambiguity

As shown in Figure 1, we started by identifying the key nouns and verbs to determine the boundary, entity, and control classes, which in turn serve as the basis for our system. These implicit and explicit requirements determine the methods and interactions among objects. Setting the boundaries from the start, through identifying ambiguities and making assumptions, would in turn prevent erroneous implementations later on.

1.2 Assumptions

For consistency in the code, we made a list of Ambiguities/Assumptions (Figure 2). Making this list allowed everyone to be on the same page, as differing interpretations of key terminology may lead to varying methods implemented, potentially resulting in unnecessary bugs.

Secondly, we assumed that all users understand the system and can navigate through the user system based on the few keywords provided to them. Hence, we did not go into details when listing the various functions in the menu system.

1.3 Features and Scope

As our goal was to implement a core feature demonstrating object-oriented principles, we decided to only implement the key functions at the start to avoid overcomplicating the system.

Core	Optional	Excluded
Basic methods to carry out operations	Good-to-have to improve user experience but not necessary	Features unsuitable for implementation now - unfeasible/too complex
- User login and authentication - Viewing of interface outputs - Functionalities specific to each use - Interface interaction i.e. register, enquiry - Intermediate checking of eligibility	- Password strength validation - graphic user interface - Notifications - password hashing - more options on tasks	- Payment handling for flats - extension to other housing services - chatbot help service
Importance: 3 - Core for operations Feasibility: 3 - Basic function interaction Timeline: 3 - Required quite immediately	Importance: 2.5 - user experience is important but less than important than core Feasibility: 2 - Varying levels of difficulty Timeline: 2 - slightly more time needed to implement	Importance: 1 - Outside of core functions Feasibility: 0.5 - operating system's scale, high cost, lack data to handle large datasets Timeline: 1 - much more time and expertise needed but not due as early/not required

Figure 3. List of core, optional, and excluded features

By grouping them this way, we were able to determine our main priorities. Core encompasses the key functionalities that must be implemented to construct a working system, playing a crucial role in the formulation of classes and their interactions.

2. System Architecture and Structural Planning

2.1 System Structure

After decomposing the problem (Figure 2), we decided to implement the MVC since each role in the entity class performs many responsibilities and compressing them together would only result in high coupling and low cohesion, associated with many dependencies.

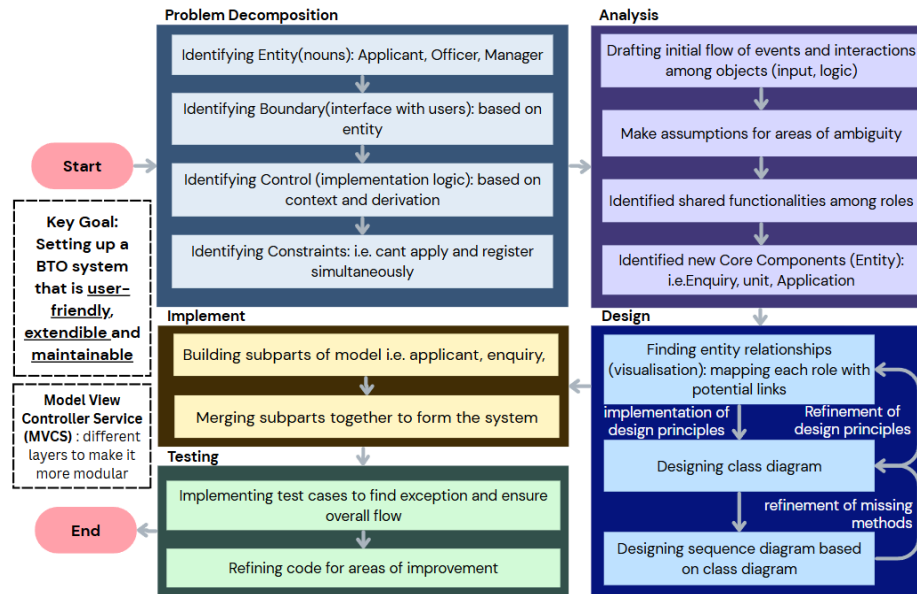


Figure 2. Flow Chart of Design of System Layout

2.2 Design Trade Off

One trade-off is balancing between simplicity and following the SOLID principles. While SOLID principles can improve the code's robustness, they increase the complexity of the number of classes. Also, including too many excluded and optional features may be unrealistic in this short period, and instead have the opposite effect. Balancing between the core and additional options reduces the likelihood of bugs emerging.

3. Object-Oriented Design

3.1 Class Diagram

Following the MVCS model and draft, we found the main entities to be **Applicant**, **HDBOfficer**, **HDBManager**, and **Project**, being heavily involved in the object interactions. Hence, we decomposed them into the MVC, where each package would focus primarily on a single task. By drawing out each entity's relationships for visualisation, we were able to determine their relationships between various class types.

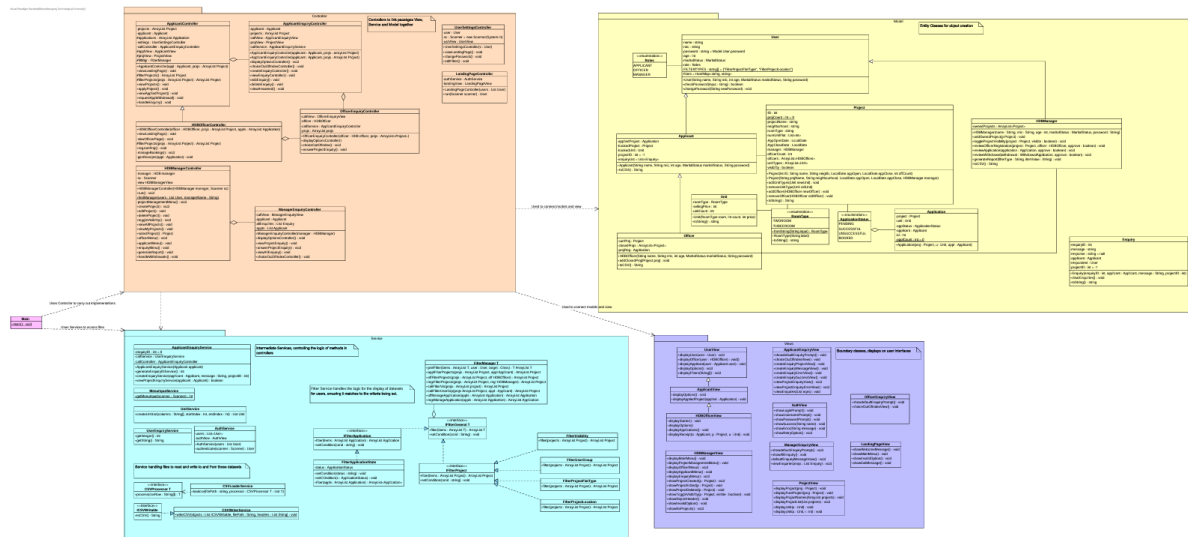
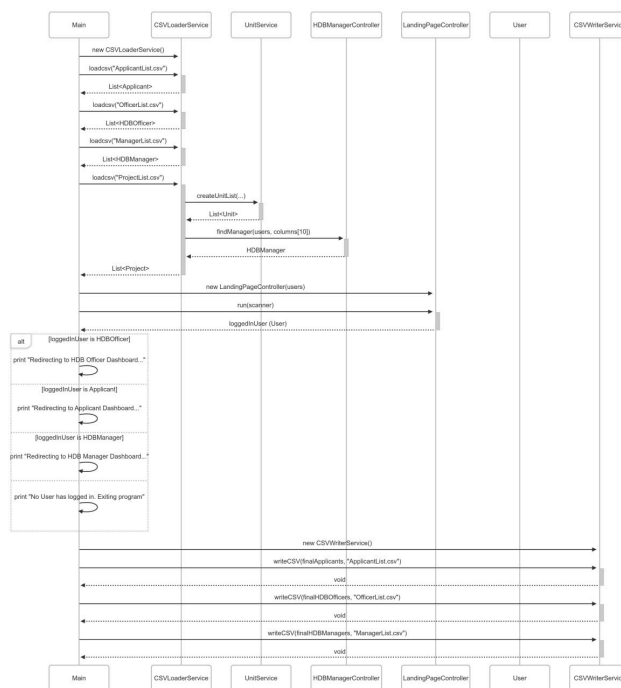
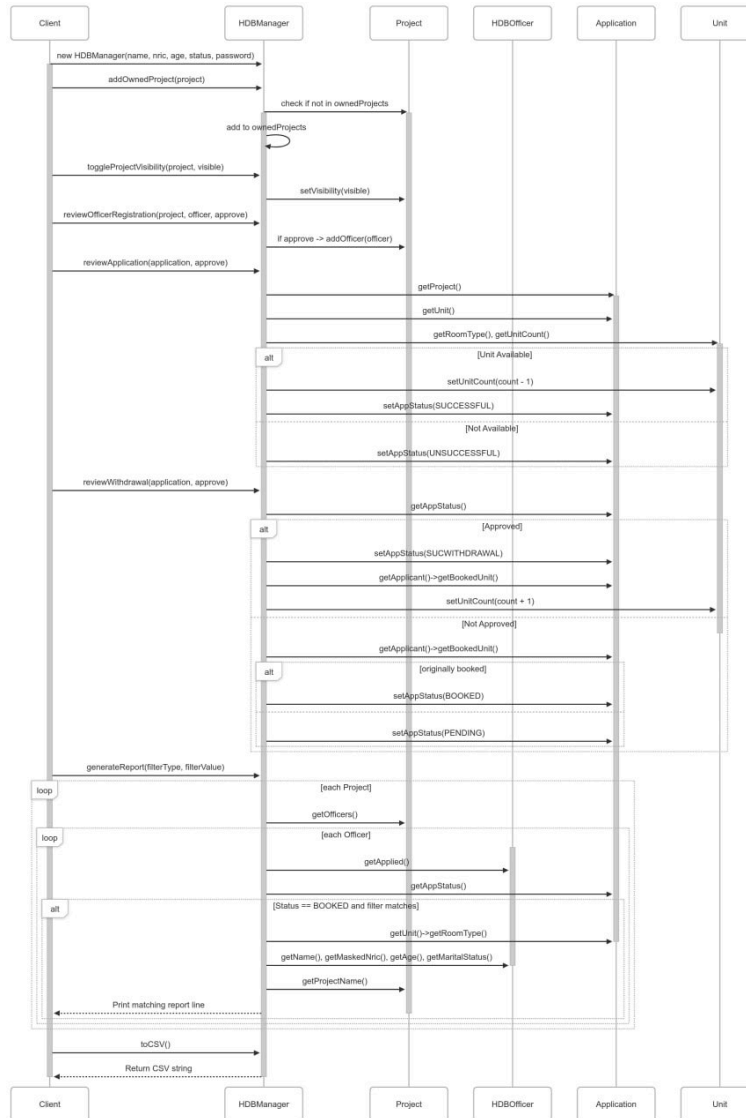


Figure 3. Class Diagram of our System's operation

3.2 Sequence Diagram



The sequence diagram shows the process from extracting the CSV files to Manager logging into their account and then saving the updated information back into the CSV files. This shows the general flow and entire picture of the complete process that each user experiences. Ideally, it should be expected for all the processes to run through smoothly without any interruptions.



From the sequence diagram, it zooms into the specific behaviour of the manager and the key features of their roles. By mapping out the process, we are able to clearly understand how information flows and identify missing methods / classes the moment one fails to link.

3.3 Application of OOD and OOP Principles

3.3.1 Single Responsibility Principle

As we implemented the MVCS (Figure 3), together with interface and enumeration, the allocation of specific responsibilities to each layer allows for high cohesion and low coupling, reducing the dependencies among classes where changes to 1 class are less likely to affect others.

3.3.2 Open-Closed Principle

Through the implementation of enums, ArrayLists, and interfaces, objects are open for extension (i.e., adding more roles, adding room types, and filter types) and closed for modification. By implementing a generic **CSVProcessor**. Being able to handle different **CSVProcessor** implementations if new files of different formatting were produced, it is open for extension. Also, it is closed for modification since no change will be needed to **CSVLoaderService** since they can easily be added onto the interface.

```
public class CSVLoaderService {

    public <T> List<T> loadcsv(String filePath, CSVProcessor<T> processor) {
        List<T> results = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
            String line;
            br.readLine(); // Skip headers

            while ((line = br.readLine()) != null) {
                String[] columns = line.split(regex:",");
                T obj = processor.process(columns);
                results.add(obj);
            }

        } catch (IOException e) {
            System.out.println("Error loading file: " + filePath);
            e.printStackTrace();
        }

        return results;
    }

    @FunctionalInterface
    public interface CSVProcessor<T> {
        T process(String[] columns);
    }
}
```

3.3.3 Liskov Substitution Principle

Our entities **Applicant**, **Officer**, **Manager** are all extended from **User**, the parent class. Using polymorphism, the subclasses are able to inherit all attributes and behaviours from **User**. Also, they can be overridden to perform their own methods without affecting the parent. For example, when extracting CSV files, we implemented `toCSV()` within each entity, allowing them to extract data from their own respective files.

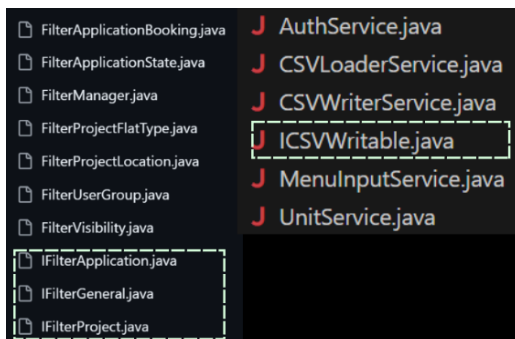
```

public class Applicant extends User implements ICSVWritable {
    public class HDBManager extends User implements ICSVWritable {
    public class HDBManager extends User implements ICSVWritable {

    @Override
    public String toCSV() {
        return String.join(delimiter:",",
            getName(),
            getNric(),
            String.valueOf(getAge()),
            String.valueOf(getMaritalStatus()),
            getPassword()

```

3.3.4 Interface Segregation Principle



As seen in our Filter service - each filter type is split up from **IFilterGeneral** into generic **IFilterProject** and **IFilterApplication**, then split further into specific Filter types. FilterManager then implements the relevant interfaces needed for the control classes.

Using ISP, we avoided the implementation of “FAT” interface where classes are forced to perform unnecessary methods. The use of 2 interfaces allows for greater flexibility in allowing more entities and classes to implement filter service.

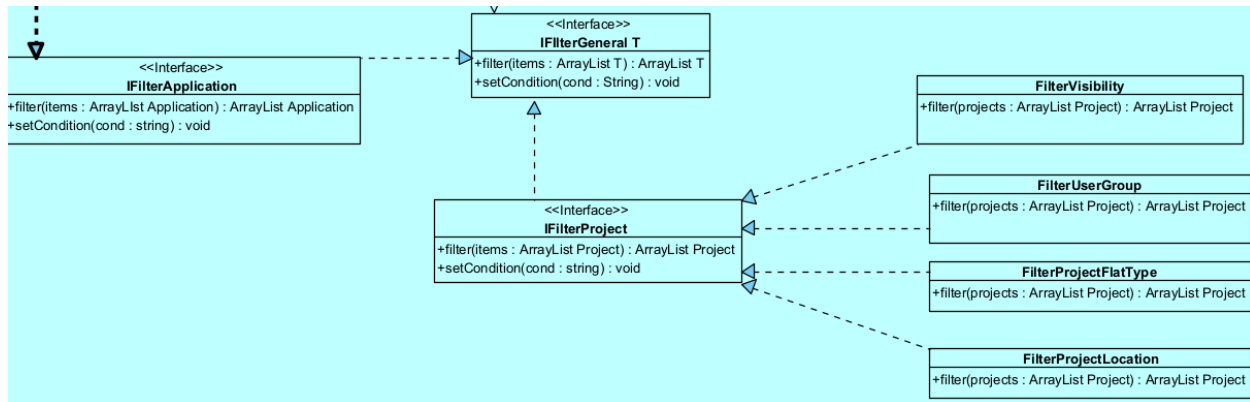
3.3.5 Dependency Inversion Principle

```

private <T> ArrayList<T> prefFilter(ArrayList<T> items, User user, Class<? extends IFilterGeneral<T>> target){
    ArrayList<T> filtered = new ArrayList<>();
    HashMap<String, String> filters = user.getFilters();

    try {
        for (String filtName : filters.keySet()) {
            Class<?> filterClass = Class.forName("src.com.BTO.Service.Filter." + filtName);
            IFilterGeneral<T> filterObj = (IFilterGeneral) filterClass.getDeclaredConstructor().newInstance();

```



By designing our filters in this manner, we are able to avoid tight coupling, enabling it to be easier to add and remove functionalities. The high level logic is being linked to the low level logic using the interface thereby decoupling them. Since abstraction depends primarily on **IFilterProject** and not the concrete classes (low level modules), we are able to separate high and low modules, where close relationships are not necessary for their implementations.

3.4 Additional OOD Principles

3.4.1 Encapsulation

For all our attributes, their state is being set to private to prevent direct access. Making use of getters and setters, they are still able to be accessed and modified indirectly without compromising on their maintainability from unexpected changes. Also we set methods to be protected if they were only needed in the subclasses but not outside packages.

```

public class Project {
    // variable declaration
    private int ID; // ID SHOULD BE SET WHEN SAVING/ LOADING
    private static int projCount = 0;

    private String projectName;
    private String neighbourhood;
    private LocalDate AppOpenDate;
    private LocalDate AppCloseDate;

    private HDManager manager;
    private int officerCount;
    private ArrayList<HDBofficer> officers = new ArrayList<HDBofficer>();
    private ArrayList<Unit> unitTypes = new ArrayList<Unit>();
    private boolean visibility;

    private ArrayList<Project> projects = new ArrayList<Project>();
    private Applicant applicant;
    protected ArrayList<Application> applications;

    // Tools
    private UserSettingsController settings;
    private ApplicantEnquiryController callController;
    protected Scanner sc = new Scanner(System.in);
    protected ApplicantView applView;
    protected ProjectView projView;
    protected FilterManager filtMgr;
  
```

3.4.2 Error Handling

By implementing exception handling throughout the code, we were able to avoid unnecessary errors and filter out invalid inputs. This allows the code to carry on without affecting the code's normal flow.

```

try {
    proj = filtered.get(choice);
}
catch(Exception e) {
    System.out.println(x:"ERROR: Invalid value!");
    return;
}

```

4. Testing

Knowing the program's implementation, we used white box testing. It involves conducting tests using known input and output values to determine if they are matched. For inputs that are based on boundary values, they are determined based on the code, ensuring that the process leads to the right outcome and not just the answer alone. Figure _ is a sample of some test cases we made.

No	Test Description	Expected output
1	(1) Login with invalid nric (2) Login with valid nric and password (3) Login with valid nric invalid password (4) Login with invalid nric invalid password	(1) Error message (2) Successfully reach landing page (3) Error message (4) Error message
2	(1) User enters a wrong current password (2) User enters a correct current password and a new password	(1) Error message (2) Password successfully updated
3	(1) For applicants, when viewing projects (2) For managers, when viewing projects (3) For officers, when viewing projects	(1) Toggled to see relevant projects based on their visibility and marital status (2) Toggled to see all projects and filtered based on their preference

4	(1) Applicant Books a flat with no previous booking (2) Applicant Books a flat with previous previous booking	(1) Error (2) Successfully Booked
5	(1) Applicant creates enquiry (2) Applicate edit enquiry (3) Applicant deletes enquiry	(1) Enquiry Successfully created (2) Enquiry Successfully Edited (3) Enquiry Successfully Deleted
6	(1) Officer registers with applied projects (2) Officer registers without any applied projects	(1) Rejected (2) Pending approval
7	(1) Officer checks their registration status	(1) Manages to see if it is pending/approved/rejected
8	(1) Create project Listing (2) Edit project Listing (3) Remove Project Listing	(1) Successfully created (2) Successfully edited (3) Successfully removed
9	(1) Manager toggles visibility	(1) Visibility is reflected in the projects that officers/applicants view

Figure 4. Test Cases

Applicant Login	Officer Login	Manager Login
<pre> --- Login --- Enter Name: John Enter Password: password Welcome, John! Redirecting to Applicant Dashboard... </pre>	<pre> --- Login --- Enter Name: Emily Enter Password: password Welcome, Emily! Redirecting to HDB Officer Dashboard... </pre>	<pre> --- Login --- Enter Name: Michael Enter Password: password Welcome, Michael! Redirecting to HDB Manager Dashboard... </pre>

From the test cases, some paths failed to run. As each path is directly linked to a particular segment of the code, we were able to pinpoint the error quickly and make necessary adjustments before using the same test to check.

5. Reflections

Our project experience gave us rich exposure to the technical and collaborative sides of system designing. One of the highlights was our success in sustaining good communication during the development phase. Meetings, lab sessions, and tutorial talks helped us develop a good rapport and ensured that any problems were sorted out early on. Tasks were distributed among the group members as per their capabilities so that the work could be divided fairly and everyone could make a significant contribution.

One of the key technical challenges that we faced was successfully applying SOLID principles. Even though we were theoretically aware of how useful these design principles are, translating them into practice, especially dealing with interface generics, proved to be more challenging than expected. There were moments when balancing flexibility and readability of code became challenging, especially while applying the Open/Closed Principle and Interface Segregation Principle. But these challenges helped to improve our understanding of abstraction and modularity in object-oriented programming.

Referring to our project concept, the use of UML diagrams in our initial planning phase was extremely useful. They gave us a clearer picture of the architecture and helped guide the development process. By envisioning class relationships and duties in advance, mentally, we were able to structure our code more sensibly and logically, adhering strictly to the MVC (Model-View-Controller) design pattern. Not only did this allow for better separation of concerns, but it helped our code become more maintainable.